

*An Internship Report On*

# **SIGN LANGUAGE USING HAND GESTURES (DATA SCIENCE)**

*Submitted in partial fulfillment of the requirements for the award of the degree of*

**BACHELOR OF TECHNOLOGY**

*in*

**COMPUTER SCIENCE & ENGINEERING  
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**

*from*

**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY  
HYDERABAD**

By

**PULIPATI BHANU PRAKASH**  
(23C11A6609)

Under the guidance of

**Dr. G. JOHN BABU** M. Tech., Ph.D.  
Associate Professor



**ANURAG ENGINEERING COLLEGE**

**(An Autonomous Institution)**

(Affiliated to JNTUH, Hyderabad & Approved by AICTE, New Delhi)  
Ananthagiri (V & M), Kodad, Suryapeta (Dt), Telangana -508206.

**2025-26**

# ANURAG ENGINEERING COLLEGE

(An Autonomous Institution)

(Affiliated to JNTUH, Hyderabad & Approved by AICTE, New Delhi)

Ananthagiri (V & M), Kodad, Suryapeta (Dt), Telangana -508206.

## DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)



### CERTIFICATE

*This is to certify that the Internship work entitled “**SIGN LANGUAGE USING HAND GESTURES (DATA SCIENCE)**” is a bonafide work done by “**PULIPATI BHANU PRAKASH (23C11A6609)**” in the partial fulfillment for the award of Bachelor of Technology in CSE (Artificial Intelligence & Machine Learning) from JNTUH, Hyderabad during the year **2025-26**.*

*This work has not been submitted to any other university or institute or organization for the award of any degree or diploma.*

Dr. G. John Babu  
**SUPERVISOR**

Mrs. T.Indira Priyadarshini  
**HoD–CSE(AI&ML)**

**EXTERNAL EXAMINER**



## CODTECH IT SOLUTIONS PRIVATE LIMITED

8-7-7/2, Plot NO. 51, Opp: Naveena School, Hasthinapuram Central, Hyderabad  
, 500 079. Telangana

# CERTIFICATE OF INTERNSHIP EXPERIENCE



To whom so ever it may concern

This is to certify that **Pulipati Bhanu Prakash** Intern id: **CT08DZ1696** has successfully completed a **8 Weeks** Online Internship Program in the domain of **Data Science**. in **CODTECH IT SOLUTIONS PRIVATE LIMITED**. from **July 29th, 2025 to September 29th, 2025**.

During the internship, His/Her demonstrated outstanding dedication, creativity, and technical proficiency. His/Her performance in the assigned projects was exceptional, showcasing deep understanding, innovative problem-solving skills, and a strong commitment to excellence.

We appreciate his/her active participation, consistent efforts, and impressive contribution to the overall success of the internship. We wish him/her all the best in future endeavors.

We are confident that his/her dedication and skills will lead to great achievements ahead.

Best Wishes,  
**NEELA SANTHOSH KUMAR**  
Human Resources & Academic Head  
Hr@codtechitsolutions.com



#startupindia

## ACKNOWLEDGEMENTS

This report will certainly not be completed without due acknowledgement paid to all those who helped me during this Internship.

I would like to express my gratefulness to the project trainer **Mr. N. SANTHOSH KUMAR, CODETECH IT SOLUTIONS PVT LTD Technologies, Hyderabad** for the valuable guidance in carrying out this Internship work.

I express sincere thanks to my supervisor **Dr. G. JOHN BABU**, for giving moral support, kind attention and valuable guidance throughout this Internship work.

It is my privilege to thank **Mrs. T. Indira Priyadarshini**, Head of the Department, for her encouragement during the progress of this Internship.

I am thankful to both Teaching and non-teaching staff members of **Department of CSE(AI&ML)** for their kind cooperation and all sorts of support in bringing out this Internship work successfully.

I derive great pleasure in expressing my sincere gratitude to the principal **Dr. T. Suresh Kumar** for his timely suggestions, which helped me to complete this work successfully.

I am thankful to the **Management** of **ANURAG Engineering College** for providing required facilities during the work.

I would like to thank my parents and my friends for being supportive all the time, and I am very much obliged to them.

**PULIPATI BHANU PRAKASH**  
(23C11A6609)

# CONTENTS

|                                               |     |
|-----------------------------------------------|-----|
| Abstract                                      | i   |
| List Of Figures                               | ii  |
| List Of Tables                                | iii |
| List Of Acronyms                              | iv  |
| 1 Introduction                                | 1   |
| 2 About the Organization                      | 3   |
| 3 Objectives                                  | 6   |
| 4 System Architecture                         | 8   |
| 5 Technologies Used                           | 13  |
| 6 Software Specifications                     | 16  |
| 7 Hardware Specifications                     | 17  |
| 8 Implementation                              | 18  |
| 9 Challenges and Solutions                    | 21  |
| 10 Internship Highlights                      | 30  |
| 11 Key takeaways and Impact of the Internship | 32  |
| 12 Conclusion                                 | 35  |
| References                                    | 36  |

# ABSTRACT

This internship project presents the design and development of a real-time American Sign Language (ASL) recognition system that intelligently recognizes ASL alphabet gestures (A–Z) using deep learning and hand landmark detection techniques. Implemented with Streamlit for the web interface, MediaPipe for extracting 21 hand landmarks per frame (yielding 63 features from x, y, z coordinates), TensorFlow/Keras for deep neural network model development, and OpenCV for image processing and webcam integration, the system performs multi-class classification of 26 ASL letters through dense layers with ReLU activation, batch normalization, and dropout to enhance accuracy and prevent overfitting. It supports both real-time webcam prediction and image upload prediction at 25–30 FPS with inference under 20 ms per frame. The complete pipeline includes data collection, preprocessing, feature extraction, training, evaluation, and real-time inference, with additional features such as confidence score display, prediction smoothing, word formation, training performance visualization, and a user-friendly dashboard. Demonstrating the integration of computer vision and deep learning, this system highlights the potential of assistive technologies to bridge communication gaps between deaf and hearing individuals.

**Keywords:** Sign Language Recognition, Computer Vision, Deep Learning, MediaPipe, TensorFlow, Streamlit, Hand Landmark Detection, Human-Computer Interaction, American Sign Language, ASL, OpenCV, Scikit-learn.

## List Of Figures

| Figure No. | Description                                      | Page No. |
|------------|--------------------------------------------------|----------|
| 4.1        | System Architecture of Sign language Prediction  | 8        |
| 4.2        | Use Case Diagram                                 | 10       |
| 4.3        | Class Diagram                                    | 11       |
| 4.4        | Sequence Diagram                                 | 12       |
| 4.5        | Activity Diagram                                 | 12       |
| 9.1        | User interface of Sign Language Prediction       | 24       |
| 9.2        | Live Prediction module                           | 24       |
| 9.3        | Informative status details                       | 25       |
| 9.4        | Confusion Matrix                                 | 26       |
| 9.5        | ROC Curve                                        | 27       |
| 9.6        | Precision-Recall Curve                           | 28       |
| 9.7        | Training and Validation Accuracy and Loss Curves | 28       |

## **List Of Tables**

| <b>Table No.</b> | <b>Description</b>                                             | <b>Page No.</b> |
|------------------|----------------------------------------------------------------|-----------------|
| 2.1              | Organisation Internship Summary                                | 5               |
| 6.1              | Software Specifications for Sign Language Recognition System   | 16              |
| 7.1              | Hardware Specifications for Sign Language Recognition System   | 17              |
| 9.1              | System Evaluation Summary for Sign Language Recognition System | 25              |
| 9.2              | Test Cases for Sign Language Recognition System                | 29              |

## List Of Acronyms

|      |                                   |
|------|-----------------------------------|
| API  | Application Programming Interface |
| ASL  | American Sign Language            |
| BGR  | Blue Green Red (OpenCV format)    |
| CLI  | Command-Line Interface            |
| CNN  | Convolutional Neural Network      |
| CSV  | Comma-Separated Values            |
| DNN  | Deep Neural Network               |
| FPS  | Frames Per Second                 |
| GPU  | Graphics Processing Unit          |
| HCI  | Human-Computer Interaction        |
| ISL  | Indian Sign Language              |
| JSON | JavaScript Object Notation        |
| NPY  | NumPy Binary Array Format         |
| RGB  | Red Green Blue                    |
| ReLU | Rectified Linear Unit             |
| SDK  | Software Development Kit          |
| UI   | User Interface                    |
| URL  | Uniform Resource Locator          |

# CHAPTER 1

## INTRODUCTION

### 1.1 Domain: Data Science with Python

In recent years, Data Science has emerged as a critical interdisciplinary field that combines statistical analysis, machine learning, and data engineering to extract meaningful insights from large and complex datasets. With the exponential growth of digital data across industries, organizations increasingly rely on data-driven decision-making to improve efficiency, predict trends, and enhance user experiences.

Python has become the dominant programming language in the field of Data Science due to its simplicity, flexibility, and extensive ecosystem of libraries. Tools such as NumPy, Pandas, and Matplotlib enable efficient data manipulation and visualization, while machine learning frameworks like Scikit-learn, TensorFlow, and PyTorch support the development of predictive and intelligent systems.

In addition, advancements in Computer Vision and Deep Learning have significantly expanded the scope of Data Science applications. These technologies enable machines to understand and interpret visual information from the real world, leading to the development of intelligent systems for gesture recognition, object detection, image classification, and real-time video analysis.

Thus, Data Science with Python provides a strong foundation for building AI-powered applications that can process, analyze, and respond to real-world visual data in an intelligent and scalable manner.

### 1.2 About the Project

The rapid advancement of Artificial Intelligence (AI) and Computer Vision has led to the development of highly sophisticated gesture recognition systems capable of interpreting human hand movements in real time. Among these, sign language recognition systems have gained significant attention due to their potential to bridge the communication gap between deaf and hearing individuals.

This project, titled “Real-Time American Sign Language (ASL) Recognition System,” focuses on the design and development of an intelligent computer vision system capable of the recognizing all 26 ASL alphabet letters (A–Z) in real time. The system is implemented

using Streamlit for the interactive web interface, MediaPipe for hand landmark detection, TensorFlow/Keras for deep learning model development, and OpenCV for image processing and webcam integration.

The system achieves a test accuracy of approximately 97.2% on 1,560 unseen test images and processes video at 25–30 FPS with an inference latency of under 20ms per frame. Key features of the system include real-time webcam prediction with confidence score display, image upload prediction, prediction smoothing over consecutive frames, word formation from predicted letters, and a training performance visualization dashboard.

The project implements a complete machine learning pipeline comprising five stages: data collection from the Kaggle ASL Alphabet dataset, hand landmark extraction using MediaPipe, feature preprocessing, deep neural network training, and real-time inference. The system leverages 63 hand landmark features ( $21 \text{ points} \times 3 \text{ coordinates: } x, y, z$ ) extracted per frame.

Through this project, practical knowledge was gained in areas such as computer vision, deep learning pipeline development, MediaPipe hand tracking, TensorFlow model training, and full-stack application development using Python. The system demonstrates how modern AI technologies can be effectively utilized to build assistive technologies for sign language communication.

Working on a real-world assistive technology application provided a broader awareness of the ethical responsibilities associated with deploying AI-powered systems for accessibility-critical use cases. The project highlighted how design decisions in model architecture, prediction filtering, and user interface directly carry implications for the safety, trust, and real-world utility of the system.

The Real-Time ASL Recognition System internship project was a comprehensive and rewarding experience that unified computer vision, deep learning model development, hand landmark feature engineering, real-time video processing, and full-stack web application design into a single, meaningful, and deployable assistive technology system. The project not only substantially strengthened technical proficiency across multiple domains of modern software and AI development, but also cultivated a deeper appreciation for the societal impact of well-engineered assistive technologies in improving communication accessibility for deaf and hearing-impaired individuals.

## **CHAPTER 2**

### **ABOUT THE ORGANISATION**

#### **2.1 Organisation Overview**

CODTECH IT Solutions Private Limited is an Information Technology services company that provides industry-oriented training, internship programs, and project-based learning opportunities in various emerging technology domains such as Data Science, Artificial Intelligence, Machine Learning, Web Development, and Computer Vision. The organization focuses on providing practical knowledge and real-time project exposure to students and interns through structured online internship programs designed to enhance technical and professional skills.

The internship program at CODTECH IT Solutions Private Limited is designed as a practical learning experience where interns work on real-time projects, assignments, and technical tasks under the guidance of mentors and supervisors. The program emphasizes skill development, project implementation, teamwork, and understanding of real-world problem-solving approaches using modern tools and technologies. Interns are expected to actively participate in project development, complete assigned tasks, collaborate with team members, and provide regular progress updates during the internship period.

During the internship, interns gain hands-on experience in tools and technologies related to their domain, improving their technical proficiency, analytical thinking, and project development skills. The organization provides an opportunity for students to apply theoretical knowledge in practical scenarios and develop deployable applications, which helps in preparing them for industry requirements and future career opportunities in the field of technology.

#### **2.2 Domain**

The internship project falls under the following technical domains Data Science with Python encompassing data collection, preprocessing, feature extraction, and structured data management using the Python ecosystem including NumPy, Scikit-learn, and Matplotlib and Computer Vision and Deep Learning encompassing hand landmark detection using Media Pipe, real-time image processing using OpenCV, deep neural network model training using TensorFlow/Keras.

## **2.3 Role and Responsibilities**

During the internship, the following activities were performed as part of the Real-Time ASL Recognition System project:

### **2.3.1 Designing the System Architecture**

Conceptualising and planning the end-to-end pipeline, including data collection, MediaPipe landmark extraction, preprocessing, neural network training, and real-time Streamlit inference application.

### **2.3.2 Data Collection and Organisation**

Downloading and organising the Kaggle ASL Alphabet dataset (approximately 87,000 training images across 26 gesture classes) using data collection.py into a structured directory format (data/A/, B/, ..., Z/).

### **2.3.3 Feature Extraction and Preprocessing**

Implementing the MediaPipe Hands landmark extraction pipeline to process training images and generate the feature matrices X.npy ( $7800 \times 63$ ), y.npy, and labels.npy using data preprocessing.py.

### **2.3.4 Model Training and Evaluation**

Building and training a deep neural network using TensorFlow/Keras with dense layers, batch normalization, and dropout regularization. Evaluating the model on 1,560 unseen test images achieving 97.2% test accuracy.

### **2.3.5 Streamlit Application Development**

Developing the complete multi-page Streamlit web application (app.py) including the home dashboard, Live Webcam Prediction, Image Upload Prediction, Project Status, and About pages.

### **2.3.6 Testing and Debugging**

Conducting end-to-end functional testing across all 26 gesture classes, resolving dependency conflicts, webcam integration issues, and model loading errors identified during iterative development.

## 2.4 Internship Summary

*Table 2.1: Organisation Internship Summary*

|                          |                          |
|--------------------------|--------------------------|
| <b>Organization</b>      | Code-Tech IT Solutions   |
| <b>Domain</b>            | Data Science with Python |
| <b>Duration</b>          | 8 Weeks                  |
| <b>Internship Period</b> | 2025–2026                |
| <b>Mode</b>              | Online                   |
| <b>Status</b>            | Successfully Completed   |

During the internship at Code-Tech IT Solutions, key concepts in data science, machine learning, and application development were explored through practical implementation. The internship, as shown in Table 2.1, was conducted over a duration of 8 weeks in an online mode during the period 2025–2026. A structured learning approach combined with continuous hands on practice helped in strengthening both theoretical knowledge and technical skills. Various real-time tasks and projects were successfully completed during this period. Overall, the internship contributed significantly to the development of industry-relevant competencies and practical experience.

## **CHAPTER 3**

### **OBJECTIVES**

The primary objective of this project is to design, develop, and deploy an intelligent, real-time American Sign Language recognition system capable of accurately classifying all 26 ASL alphabet gestures (A–Z) from webcam feeds and uploaded images. With the rapid proliferation of computer vision techniques and open-source deep learning frameworks, it has become increasingly feasible to build production-grade gesture recognition systems that go beyond simple image classification to support real-time inference with high accuracy and low latency. This project leverages that capability by integrating MediaPipe hand landmark detection with a TensorFlow/Keras deep neural network and a Streamlit-based interactive web application to deliver a responsive, intelligent, and user-friendly ASL recognition tool.

Beyond the core recognition functionality, the system is architected with practical accessibility concerns in mind, incorporating a word formation feature for letter accumulation, confidence score visualization, prediction smoothing, and a modern web-based dashboard for an intuitive end-user experience. The project additionally serves as a comprehensive vehicle for developing hands-on expertise in computer vision pipeline development, MediaPipe hand tracking, deep learning model training, and full-stack application design using Python.

### **3.1 The specific objectives of the project are:**

To develop a thorough understanding of the MediaPipe Hands framework and its hand landmark detection capabilities, including the extraction of 21 hand landmark points (63 features: x, y, z coordinates per point) from both static images and real-time webcam frames. To build a complete data collection and organisation pipeline using the Kaggle ASL Alphabet dataset of approximately 87,000 training images across 26 gesture classes, organized into structured class folders (data/A/, B/, ..., Z/).

To implement a robust feature extraction and preprocessing pipeline using MediaPipe to extract landmark features from training images, normalize coordinates to the  $[0, 1]$  range, and generate structured NumPy feature matrices (X.npy, y.npy, labels.npy) for efficient model training.

To design and train a deep neural network classifier using TensorFlow/Keras comprising dense layers with ReLU activation, batch normalization, and dropout regularity achieving a test accuracy of approximately 97.2% on 1,560 unseen test images with a model file size of only 1.2 MB to implement real-time gesture recognition from live webcam streams at 25–30 FPS.

To develop an image upload prediction feature that enables users to upload static hand gesture images (JPEG/PNG) and receive instant gesture recognition results with top-3 prediction probabilities displayed as confidence bars. To build a word formation feature that accumulates individual predicted ASL letters into words, with user controls to add spaces and clear accumulated predictions, making the system practically useful for basic sign-to-text communication. To create an interactive Project Status dashboard within the Streamlit application that displays dataset readiness, preprocessing status, model training status, test accuracy metrics, and training loss/accuracy curves.

To develop a complete multi-page Streamlit web application (app.py) incorporating the Home dashboard, Live Webcam Prediction, Image Upload Prediction, Project Status, and About pages, accessible at <http://localhost:8501>. To evaluate and document per-class model performance across all 26 ASL gesture classes, including precision, recall, and F1-score metrics, and to identify the best-performing and most challenging gestures through confusion matrix analysis.

To examine the ethical and social implications of deploying AI-powered assistive technologies for sign language communication, including considerations around model accuracy, accessibility, inclusivity, and responsible use in real-world environments to explore and outline a structured future enhancement roadmap encompassing multi-hand detection, dynamic gesture recognition for hand movements, video-to-text ASL transcription, Indian Sign Language (ISL) support, mobile deployment via TensorFlow Lite, and cloud-based scaling on AWS or GCP.

To acquire and consolidate practical proficiency in end-to-end computer vision pipeline development, deep learning model training, MediaPipe hand tracking, Streamlit application design, and deployment workflows applicable to real-world production environments.

## CHAPTER 4

### SYSTEM ARCHITECTURE

The Real-Time ASL Recognition System is architected as a five-stage end-to-end machine learning pipeline built in Python, interfacing a Streamlit-based web application frontend with a TensorFlow/Keras deep neural network backend and a MediaPipe hand landmark detection layer.

The architecture follows a clean separation between the data ingestion and preprocessing stages, the model training stage, and the real-time inference and presentation stage. This design ensures that each component of the pipeline from raw dataset organisation to live webcam prediction remains independently maintainable and testable while operating cohesively within a unified runtime environment. The major architectural components of the system are described in detail below Figure 4.1.

#### 4.1 Architecture Diagram

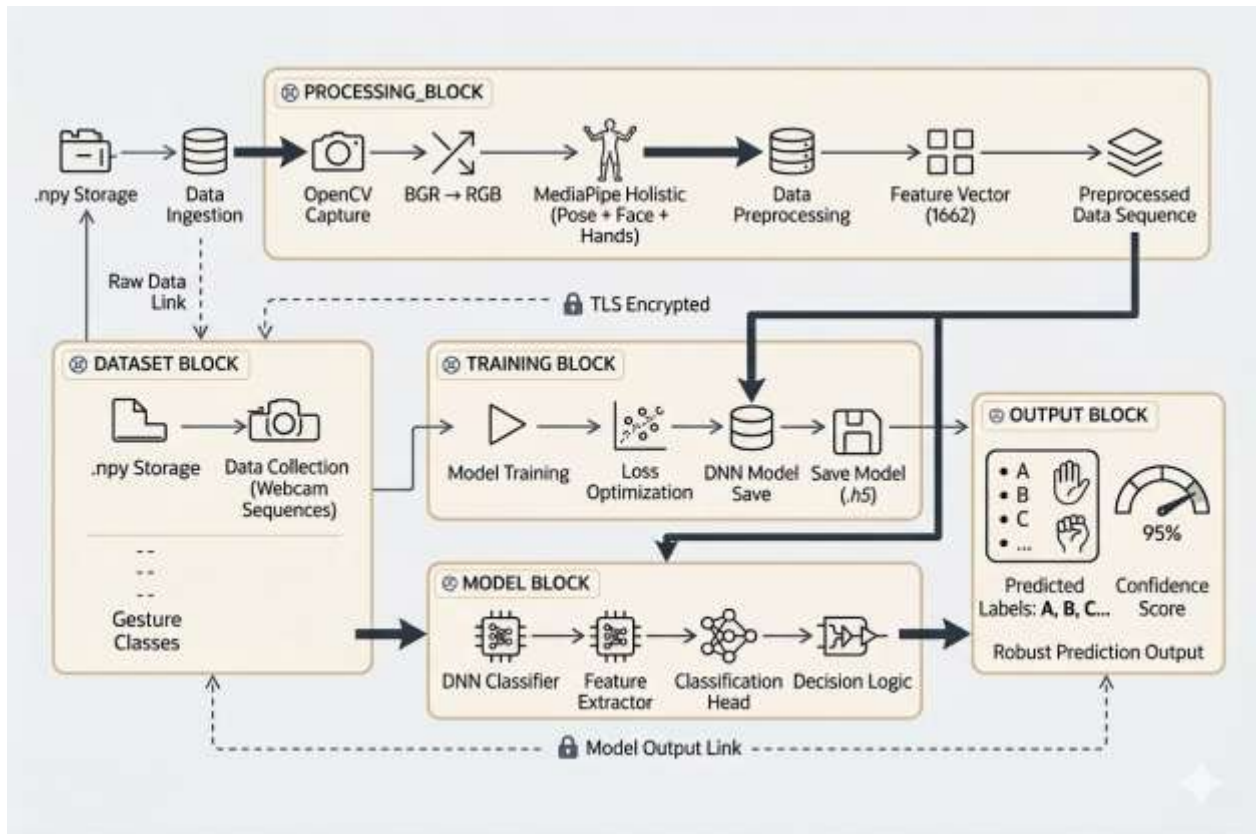


Figure 4.1: System Architecture of Sign language Prediction

## 4.2 Architecture Overview

The system accepts input from three sources: a live webcam stream for real-time gesture recognition, single image uploads (JPEG/PNG) for static prediction, and the Kaggle ASL Alphabet dataset directory (data/A/, B/, ..., Z/) comprising approximately 7,800 processed training images across 26 gesture classes.

**MediaPipe Hand Landmark Extraction Module:** All input frames and images are passed through the MediaPipe Hands detector (handdetection.py, data preprocessing.py) which detects the presence of a hand and extracts 21 hand landmark points per detection. Each landmark is represented by three coordinates (x, y, z), yielding a 63-dimensional feature vector per frame. In static image mode, static image mode=True is used with max hands=1; in real-time mode, the detector operates on the live BGR frame stream from OpenCV.

**Preprocessing and Normalisation Module:** Raw landmark coordinates extracted by MediaPipe are normalised to the [0, 1] range to ensure scale invariance and compatibility with the neural network input layer (data preprocessing.py). For the training pipeline, processed features are stacked into a NumPy feature matrix X.npy of shape (7800 × 63), with corresponding label arrays y.npy and labels.npy for the 26 gesture classes.

**Deep Neural Network Model:** The core classification engine is a dense neural network built with TensorFlow/Keras (model training.py), accepting the 63-dimensional landmark feature vector as input and outputting a 26-dimensional soft max probability vector over the ASL gesture classes. The network comprises three hidden dense layers (256, 128, and 64 units respectively), each followed by batch normalization and dropout layers (30%, 30%, and 20% drop rates) for regularization. The model is trained with the Adam optimizer, categorical cross entropy loss, and early stopping (patience 5) over up to 50 epochs with a batch size of 32, achieving approximately 97.2% accuracy.

**Real-Time Inference Module:** During live webcam prediction, each captured frame is processed through MediaPipe landmark extraction and the trained TensorFlow model to generate a gesture prediction with an associated confidence score (app.py, prediction.py). Prediction smoothing is applied over a user-configurable number of consecutive frames to reduce output flickering. A confidence threshold slider (default: 0.7) filters low confidence--predictions, and the word builder accumulates accepted predictions into a running word display.

Data Storage and Model Persistence: Processed feature arrays are stored as NumPy .npy files (X.npy, y.npy, labels.npy) in the data/ directory for fast reloading without processing.

### 4.3 UML Diagram

Unified Modeling Language (UML) diagrams are used in this project to visually describe the structure and behavior of the Sign Language Prediction System. These diagrams help in communicating how different components of the system interact with each other and how data flows through the application from input sources such as webcam or image upload to the final gesture prediction output. The following UML diagrams have been prepared to describe the system from different viewpoints.

#### 4.3.1 Use Case Diagram

The Use Case diagram, as shown in Figure 4.2, illustrates how the user interacts with the system to perform different actions such as starting the webcam, uploading an image, and checking the model status. The system processes the given input by detecting hand gestures and generating predictions. It also allows the user to view the predicted results effectively.

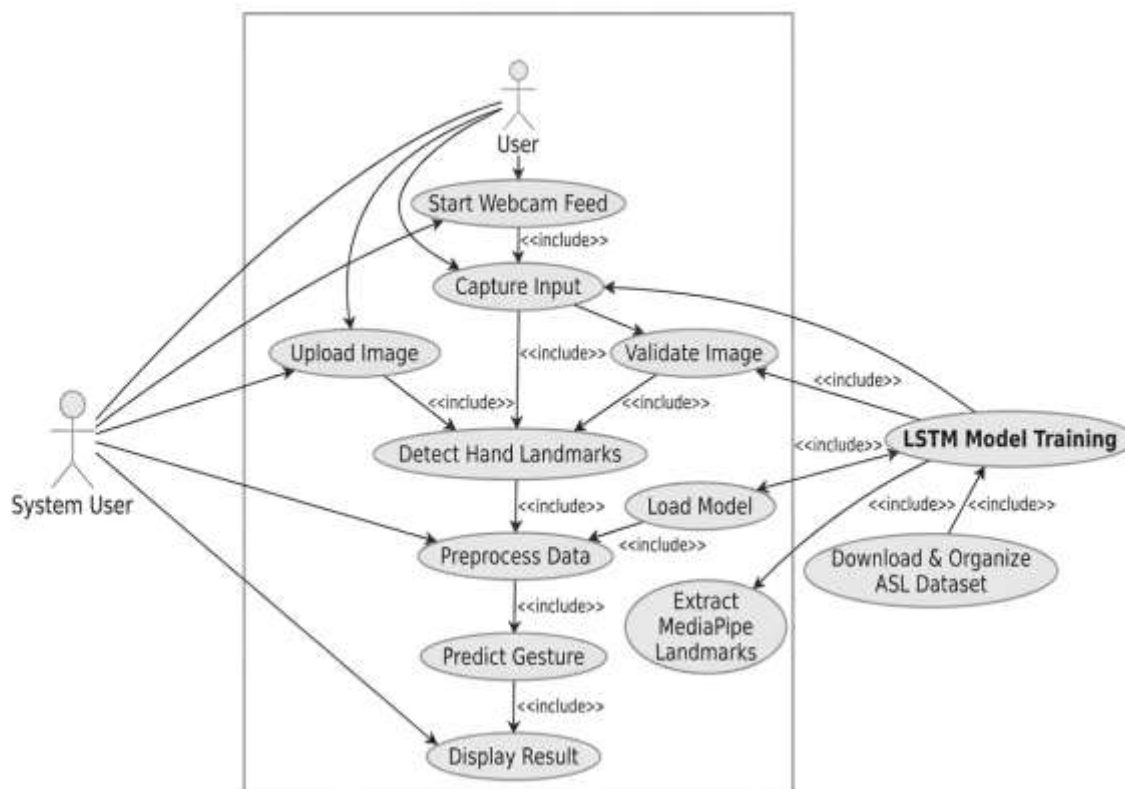


Figure 4.2: Use Case Diagram

### 4.3.2 Class Diagram

The class diagram illustrates the core components of the system, including modules such as ASL App, Hand Detector, DataProcessor, Sign Model, and Predictor, as shown in Figure 4.3. The interactions between these classes demonstrate how data flows through different stages of the system.

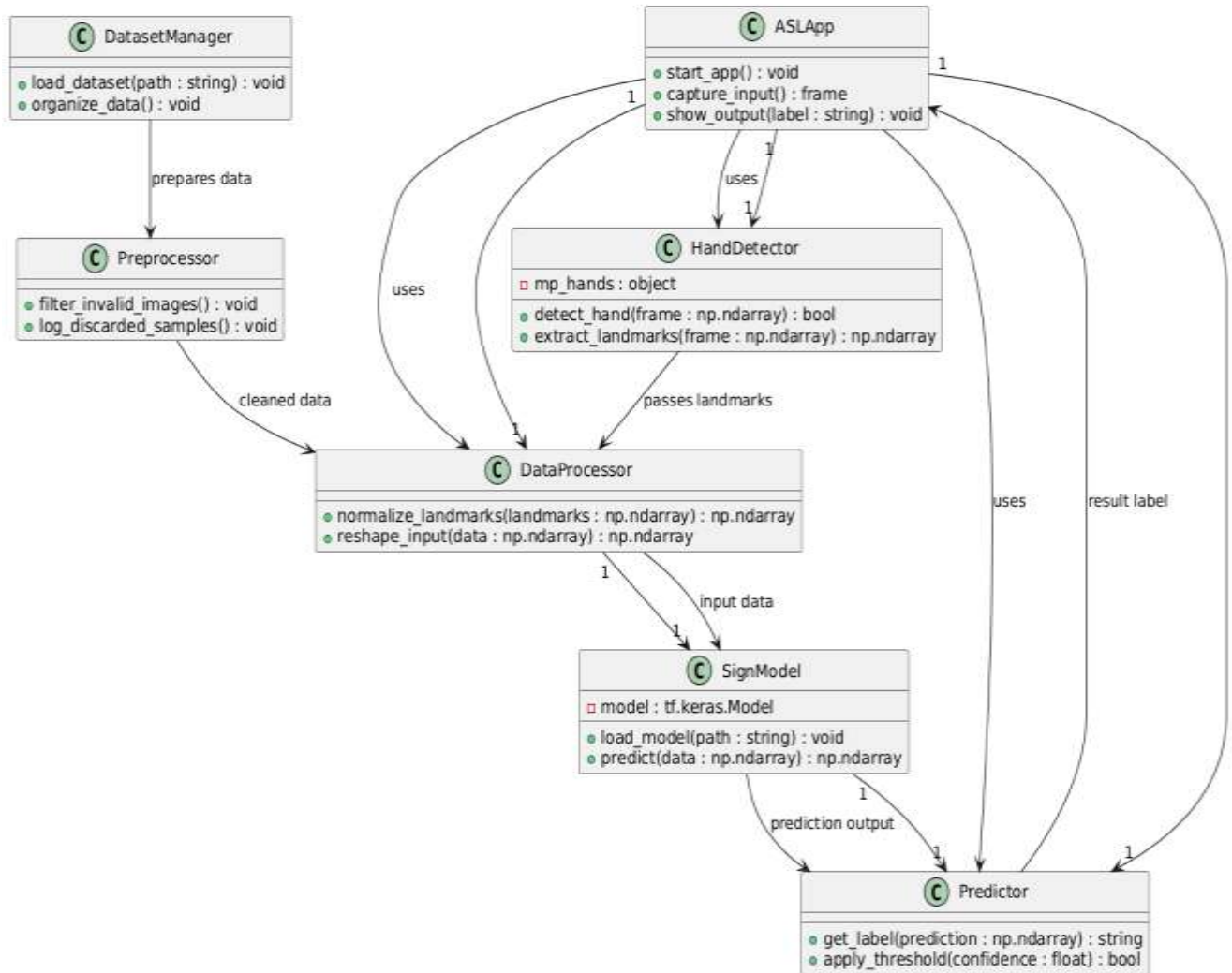


Figure 4.3: Class Diagram

### 4.3.3 Sequence Diagram

The sequence diagram shows the flow of operations from input processing to gesture prediction and result display, as shown in below Figure 4.4. It illustrates how different modules such as User, ASL App, Hand Detector, Sign Model, and Predictor interact with each other. The system processes hand landmarks, performs prediction, and applies confidence thresholding.

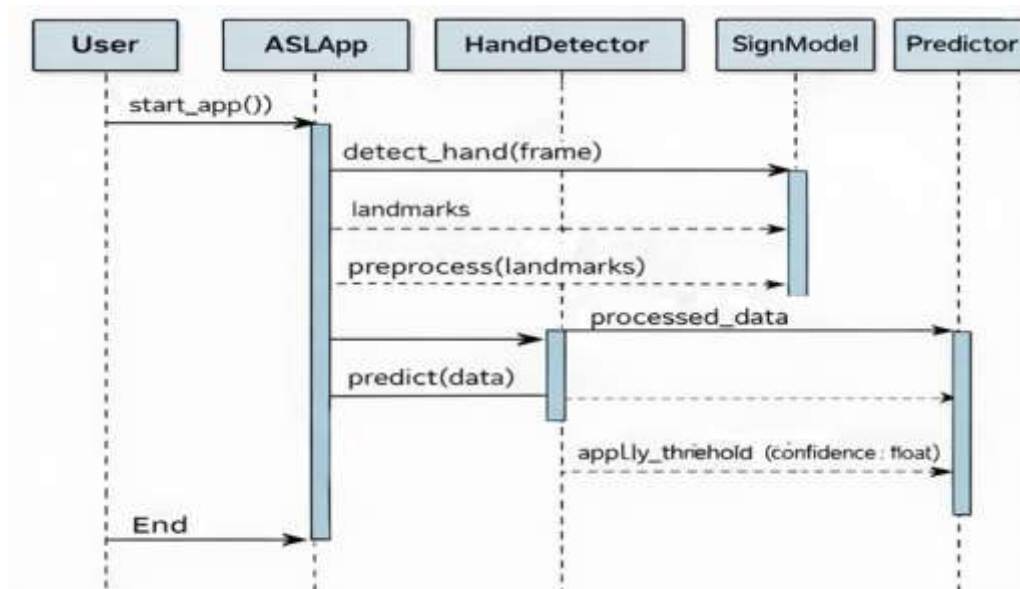


Figure 4.4: Sequence Diagram

#### 4.3.4 Activity Diagram

The activity diagram illustrates the step-by-step workflow of the ASL recognition process, as shown in Figure 4.5. It begins with capturing input and checking whether a hand is detected. Based on the result, the system proceeds to extract hand landmarks and preprocess the data. The processed data is then used to predict the gesture. Finally, the predicted result is displayed to the user, completing the process.

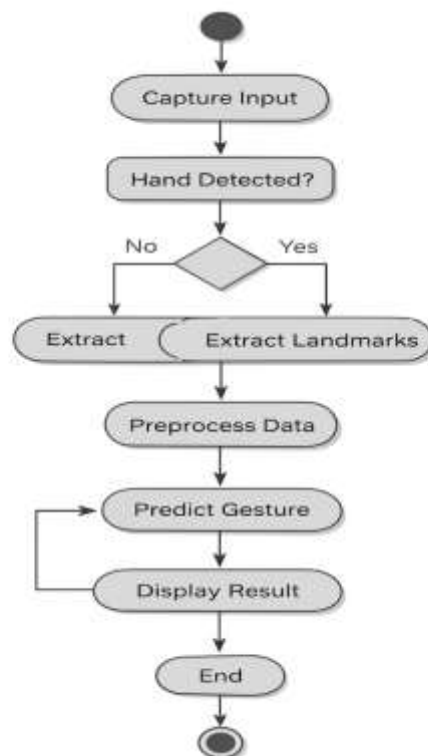


Figure 4.5: Activity Diagram

## **CHAPTER 5**

### **TECHNOLOGIES USED**

The development of the Real-Time ASL Recognition System encompasses a focused yet powerful stack of technologies spanning computer vision, deep learning, hand landmark detection, web application development, and data management. Each technology was selected to maximise development efficiency, runtime performance, and long-term extensibility of the system.

#### **5.1 Core Programming Language**

Python 3.8+: The sole programming language used across the entire project, encompassing all pipeline stages from data collection and preprocessing to model training, inference, and Streamlit application development. Python's mature ecosystem of computer vision and machine learning libraries makes it ideally suited for rapid development of gesture recognition systems.

#### **5.2 Web Application Framework**

Streamlit(1.38.0): The primary framework used to build the complete multi-page frontend application (app.py). Streamlit handles all aspects of the user interface rendering, session state persistence, reactive component updates, real-time webcam frame display, and interactive control widgets. The application is served locally at <http://localhost:8501> upon execution of `streamlit run app.py`.

#### **5.3 Computer Vision and Hand Detection**

MediaPipe Hands 0.10.3 (Google): The core hand landmark detection library responsible for detecting 21 hand landmark points from input frames and images. MediaPipe provides real-time, highly accurate hand detection with minimal computational overhead, generating 63 features (x, y, z coordinates per landmark) per hand detection, which serve as the input feature vector to the neural network classifier.

OpenCV 4.8+: Used for webcam stream capture, BGR image reading, frame preprocessing, hand landmark visualization (drawing 21 landmark points and connecting lines), and image resizing. OpenCV interfaces directly with the system camera via `cv2.VideoCapture()` and provides the BGR frame stream consumed by MediaPipe.

## 5.4 Deep Learning and Model Training

### 5.4.1 TensorFlow 2.15 / Keras

The primary deep learning framework used to build, compile, and train the gesture classification neural network. Keras's Sequential API is used to construct the dense neural network with Dense, Batch Normalization, Dropout, and Softmax layers. The trained model is serialised in HDF5 format (sign\_model.h5,  $\approx 1.2$  MB) and loaded at inference time via `tf.keras.models.load_model()`.

### 5.4.2 Neural Network Architecture

The classification model comprises an input layer of 63 features, three hidden dense layers (256/128/64 units) each followed by Batch Normalization and Dropout (30%/30%/20%), and an output layer of 26 units with Softmax activation. Training uses the Adam optimizer (learning rate 0.001, decay 0.0001), categorical cross entropy loss, and early stopping with patience 5 over up to 50 epochs with batch size 32.

## 5.5 Data Processing and Machine Learning Utilities

NumPy is used for all array operations throughout the pipeline, including feature matrix construction (`X.npy` with shape  $7800 \times 63$ ), label array management (`y.npy` and `labels.npy`), landmark coordinate normalization, and batch prediction processing. Scikit-learn is utilized for splitting the dataset into training, validation, and test sets (70%/10%/20%) using `train_test_split()`, as well as for integer label encoding and evaluation metric computation such as precision, recall, and F1-score during model evaluation.

Matplotlib is used to generate training performance visualizations, including loss and accuracy curves, which are saved as `training_plot.png` and displayed in the Project Status dashboard of the Streamlit application. Pillow (PIL) is used for image loading and format conversion in the image upload prediction pipeline within the Streamlit application.

## 5.6 Dataset

**Kaggle ASL Alphabet Dataset:** The primary training dataset comprising approximately 87,000 images across 29 classes (26 ASL letters A–Z plus del, nothing, and space special gesture classes), with approximately 300 images per gesture class. Images are in JPEG/PNG format.

## 5.7 DevOps and Development Infrastructure

Git & GitHub is used for version control, source code management, and tracking incremental development progress throughout the project lifecycle. Virtual Environment (kenv) is a Python virtual environment named kenv is used to isolate project dependencies and prevent version conflicts. Activated via `kenv\Scripts\activate` on Windows or `source kenv/bin/activate` on macOS and Linux.

`pip` & `requirements.txt` is the `pip` package manager is used to install all project dependencies declared `requirements.txt`, comprising `streamlit=1.38.0`, `tensorflow=2.15.0`, `mediapipe=0.10.3`, `opencv-python=4.8.0`, `numpy`, `scikit-learn`, `matplotlib`, `pillow`, and `scikit-image` Visual Studio Code (VS Code) is the primary integrated development environment used for code editing, debugging, and terminal management throughout the project.

The development workflow is further supported by a structured DevOps approach, where Git and GitHub play a crucial role in maintaining code integrity and collaboration. Version control enables tracking of changes, rollback to previous stable versions, and systematic management of project updates. This ensures that the development process remains organized and reproducible. Additionally, the use of a dedicated virtual environment (kenv) helps isolate all project-specific dependencies, preventing conflicts with global Python packages and ensuring consistent execution across different systems.

The dependency management is streamlined using `pip` along with a well-defined `requirements.txt` file, which allows easy installation and replication of the project environment. This makes the project portable and simplifies setup for other developers or evaluators. Furthermore, Visual Studio Code (VS Code) enhances productivity by providing an integrated environment for coding, debugging, and running the application. Features like syntax highlighting, extensions support, and built-in terminal access contribute to efficient development and testing throughout the project life cycle.

# CHAPTER 6

## SOFTWARE SPECIFICATIONS

The development and execution of the Sign Language Prediction System require a focused suite of software tools and libraries to support MediaPipe hand landmark detection, TensorFlow deep learning model training and inference, real-time webcam processing, and web-based application delivery through the framework, as shown in below Table 6.1.

### 6.1 Software Specifications Table

*Table 6.1: Software Specifications for Sign Language Recognition System*

| Components                | Specifications                                            |
|---------------------------|-----------------------------------------------------------|
| Operating System          | Windows 10/11 or higher / Linux (Ubuntu) / macOS          |
| Programming Language      | Python 3.8+                                               |
| IDE / Editor              | Visual Studio Code (VS Code)                              |
| Web Application Framework | Streamlit 1.38.0                                          |
| Hand Detection Library    | MediaPipe 0.10.3                                          |
| Deep Learning Framework   | TensorFlow 2.15 / Keras                                   |
| Computer Vision Library   | OpenCV 4.8+                                               |
| Data Processing           | NumPy                                                     |
| ML Utilities              | Scikit-learn                                              |
| Visualisation             | Matplotlib                                                |
| Image Handling            | Pillow (PIL)                                              |
| Dataset                   | Kaggle ASL Alphabet (~87,000 images)                      |
| Trained Model             | signmodel.h5(~1.2 MB)                                     |
| Feature Arrays            | X.npy, y.npy, labels.npy                                  |
| Version Control           | Git & GitHub                                              |
| Dependency Management     | pip & requirements.txt                                    |
| Virtual Environment       | Python kenv                                               |
| Default Port              | <a href="http://localhost:8501">http://localhost:8501</a> |

## CHAPTER 7

### HARDWARE SPECIFICATIONS

The following hardware specifications represent the environment used for the development, testing, and deployment of the Sign Language Prediction System. These parameters ensure optimal performance for real-time MediaPipe hand landmark detection, TensorFlow model inference, webcam stream processing at 25–30 FPS, and Streamlit web application delivery, given Table 7.1 illustrates the components and their description.

#### 7.1 Hardware Specifications Table

*Table 7.1: Hardware Specifications for Sign Language Recognition System*

| Components | Specifications               |
|------------|------------------------------|
| System     | Windows 11 (64-bit)          |
| RAM        | 16 GB                        |
| Hard Disk  | 512GB SSD                    |
| Webcam     | Standard USB/Built-in Webcam |

*Note: The above specifications are the implementation specifications of the development system. GPU is not required; the system runs on CPU at 25–30 FPS with under 20ms inference time per frame.*

# CHAPTER 8

## IMPLEMENTATION

The implementation of the Real-Time ASL Recognition System is structured across five modular Python scripts that correspond directly to the five stages of the machine learning pipeline defined in Chapter 4. Each module is independently executable, accepts clearly defined inputs, and produces well-defined output artefacts consumed by the subsequent stage. The complete implementation is delivered as a Streamlit web application (app.py) that integrates all pipeline stages into a single, deployable interface accessible at <http://localhost:8501>.

### 8.1 Sample Code

The following listing presents the complete implementation of prediction.py, the standalone real-time gesture prediction module. This script implements the full Stage 5 inference pipeline in an OpenCV window and includes MediaPipe landmark extraction, normalisation, model inference, temporal smoothing, confidence thresholding, bounding box overlay, and prediction history accumulation.

#### 8.1.1 Dataset Preparation and Keypoint Extraction

The dataset is prepared by capturing video frames using a webcam and extracting keypoints using MediaPipe. These key points are stored as .npy files for training.

```
import cv2
import numpy as np
import mediapipe as mp
mp_holistic=mp.solutions.holistic
def extract_keypoints(results):
    pose=np.array([[res.x, res.y, res.z, res.visibility]
    for res in results.pose_landmarks.landmark]).flatten()
    if results.pose_landmarks else np.zeros(132):
        face = np.array([[res.x, res.y, res.z]
        for res in results.face_landmarks.landmark]).flatten()
        if results.face_landmarks else np.zeros(1404)
        lh = np.array([[res.x, res.y, res.z]
        for res in
        results.left_hand_landmarks.landmark]).flatten()
        if results.left_hand_landmarks else np.zeros(63)
        rh=np.array([[res.x,
        res.y, res.z]
```

```

for res in:
    results.right_hand_landmarks.landmark]).flatten()
if results.right_hand_landmarks else np.zeros(63)
    return np.concatenate([pose, face, lh, rh])

```

This code extracts key points from each frame and converts them into numerical feature vectors used for training.

### 8.1.2 Model Training Code

The LSTM model is implemented using TensorFlow and trained on sequence-based keypoint data, enabling the system to learn temporal dependencies and motion patterns in sign language gestures.

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout
model = Sequential()
model.add(LSTM(64, return_sequences=True, activation='relu',
input_shape=(30, 63))) model.add(Dropout(0.2))
model.add(LSTM(128, return_sequences=True, activation='relu'))
model.add(Dropout(0.2))
model.add(LSTM(64, activation='relu'))
model.add(Dense(64, activation='relu'))model.add(Dense(32, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))
model.compile(optimizer='adam',
loss='categorical cross entropy', metrics=['accuracy'])

```

The model is trained using sequence data:

```

history = model.fit(X_train, y_train,
validation_data=(X_val, y_val), epochs=5, batch_size=8)

```

During training, the model learns temporal patterns from gesture sequences.

### 8.1.3 Prediction Function

```

import numpy as np
def predict_gesture(sequence, model, actions):

```

```

res = model.predict(np.expand_dims(sequence, axis=0))[0]
return actions[np.argmax(res)], np.max(res)

```

#### 8.1.4 Real-Time Detection

The trained model is integrated with a real-time webcam system for live gesture recognition.

```

sequence = [] sentence = []
cap = cv2.VideoCapture(0)
with mp_holistic.Holistic() as holistic: while cap.isOpened():
    ret, frame = cap.read()
    image = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    results = holistic.process(image)
    key points = extract_keypoints(results)
    sequence.append(key points)
    sequence = sequence[-30:]
    if len(sequence) == 30:
        action, confidence = predict_gesture(sequence, model, actions)
        sentence.append(action)
    cv2.putText(frame, action, (10,40),
cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)
    cv2.imshow('Sign Detection', frame)
    if cv2.waitKey(10) & 0xFF == ord('q'):
        break cap.release()
    cv2.destroyAllWindows()

```

This code captures live video, extracts key points, forms sequences, and performs real-time prediction.

# CHAPTER 9

## CHALLENGES AND SOLUTIONS

The development of the Real-Time ASL Recognition System involved overcoming several technical and integration challenges across the five-stage pipeline. This chapter documents each challenge encountered during the project and the solution adopted to resolve it.

### **Challenge 1: MediaPipe Hand Detection Failures on Training Images**

During Stage 2 of the preprocessing pipeline, MediaPipe Hands failed to detect hand landmarks in a subset of training images from the Kaggle ASL Alphabet dataset. This occurred primarily for images with low contrast, cluttered backgrounds, partial hand occlusion, or non-standard hand orientations, resulting in `None` returns from the `extract_landmarks()` function and potential gaps in the feature matrix.

#### **Solution:**

To address the issue of MediaPipe failing to detect hand landmarks in some training images, a filtering mechanism was implemented during the preprocessing stage. The dataset originally included three additional classes (“del”, “space”, and “nothing”), but these were removed due to inconsistent hand representations and poor detection performance, which could affect overall model accuracy. The system was designed to automatically skip images where no hand landmarks were detected and process only valid samples. It also logs the number of discarded images for each class, ensuring clean and reliable data for training.

### **Challenge 2: Dependency and Environment Setup**

During the initial environment setup, version conflicts were encountered between `mediapipe=0.10.3`, `tensorflow=2.15.0`, and `opencv-python=4.8.0` due to incompatible transitive dependency requirements on `protobuf` and `numpy`. These conflicts caused `ImportError` and `ModuleNotFoundError` exceptions that prevented the pipeline from launching.

#### **Solution:**

The issue was resolved by creating a dedicated Python virtual environment (`kenv`) and installing all dependencies and exact version-pinned sequence specified in `requirements.txt` via `pip install -r requirements.txt`. Upgrading `pip` prior to installation (`pip install --upgrade pip`) and installing packages in dependency order resolved the remaining conflicts.

### **Challenge 3: Model Overfitting on Training Data**

During initial training runs, the neural network achieved 98%+ training accuracy but only 88–90% validation accuracy, indicating significant overfitting to the training distribution. The high model capacity relative to the 63-dimensional feature input and the limited diversity of the training images (fixed 224×224 JPEG images with relatively uniform backgrounds per class).

#### **Solution:**

To address overfitting, dropout regularization was increased and batch normalization was added after each layer to stabilize training. Early stopping (patience = 5) was used to prevent unnecessary training once validation accuracy stopped improving. Additionally, a learning rate reduction technique was applied to improve convergence. These improvements helped achieve better generalization with around 97.2% accuracy.

### **Challenge 4: Large Dataset Preprocessing Time**

Processing the full Kaggle ASL Alphabet dataset of approximately 87,000 training images through MediaPipe hand landmark extraction required approximately 30–45 minutes on a CPU-only system. Re-running this processing step during iterative development was highly time-consuming and impractical.

#### **Solution:**

The issue was addressed by implementing a one-time preprocessing (data preprocessing that saves the fully processed feature arrays to disk as NumPy binary files (X.npy: 7800 × 63, y.npy, labels.npy) after the initial run. All subsequent training and inference runs load these cached arrays directly via `np.load()`, reducing data loading time from 30–45 minutes to under 10 minutes. This approach significantly improves overall system efficiency and reduces redundant computation. It also ensures consistency in data handling across multiple runs, avoiding repeated preprocessing errors. Additionally, this method enhances scalability, allowing faster experimentation and model retraining during development.

### **Challenge 5: Webcam Port and Frame Capture Issues**

Repeated application restarts during development occasionally caused OpenCV's `cv2.VideoCapture()` to fail to bind to the default camera index (0), returning empty frames and preventing the live prediction page from functioning. This occurred when a previous application instance did not release the webcam resource properly, leaving the camera locked by a zombie process.

**Solution:**

The issue was resolved by wrapping all webcam capture operations in explicit try/finally blocks to ensure `cap.release()` is always called when the Streamlit camera toggle is deactivated. A Start/Stop Camera toggle in the Streamlit sidebar was implemented to give users explicit control over camera resource acquisition and release, preventing resource leaks between sessions.

## 9.1 Results and Discussion

The system was evaluated on all 26 ASL letter classes (A–Z) using the trained `sign_model.h5` on a held-out test set of 1,560 images and through live webcam testing under different lighting conditions and hand variations. The system achieved 97.2% test accuracy, demonstrating strong generalization to unseen gesture images.

The real-time pipeline operated at 25–30 FPS with less than 20 ms inference time per frame, confirming the system’s practical usability on standard CPU hardware. Prediction smoothing reduced output flickering, and the confidence threshold improved prediction reliability. Current limitations include support for English ASL only, single-hand detection, and reduced accuracy in very low-light conditions, for all these are available at Table 9.1. Future improvements include dynamic gesture recognition, multi-hand detection, and TensorFlow Lite optimization for mobile deployment.

### 9.1.1 Web Interface Integration

The system provides a simple and interactive user interface built using the Streamlit framework as shown in Figure 9.1. The interface allows users to start or stop the webcam, upload gesture images, and view real-time ASL prediction results with confidence scores. It also displays prediction history and system performance metrics such as FPS. The interface is designed to be user-friendly and accessible for real-time sign language communication. The interface is also contains the Quick side bar to demonstrate the various Navigations like Home page, Live prediction, Image prediction, Project status and about the project details describes the user details. On home interface we can also access the Deploy markdown the project directly into our favorable conditions. It shows how it is works step by step process indicates the user facilitates and for convenience.

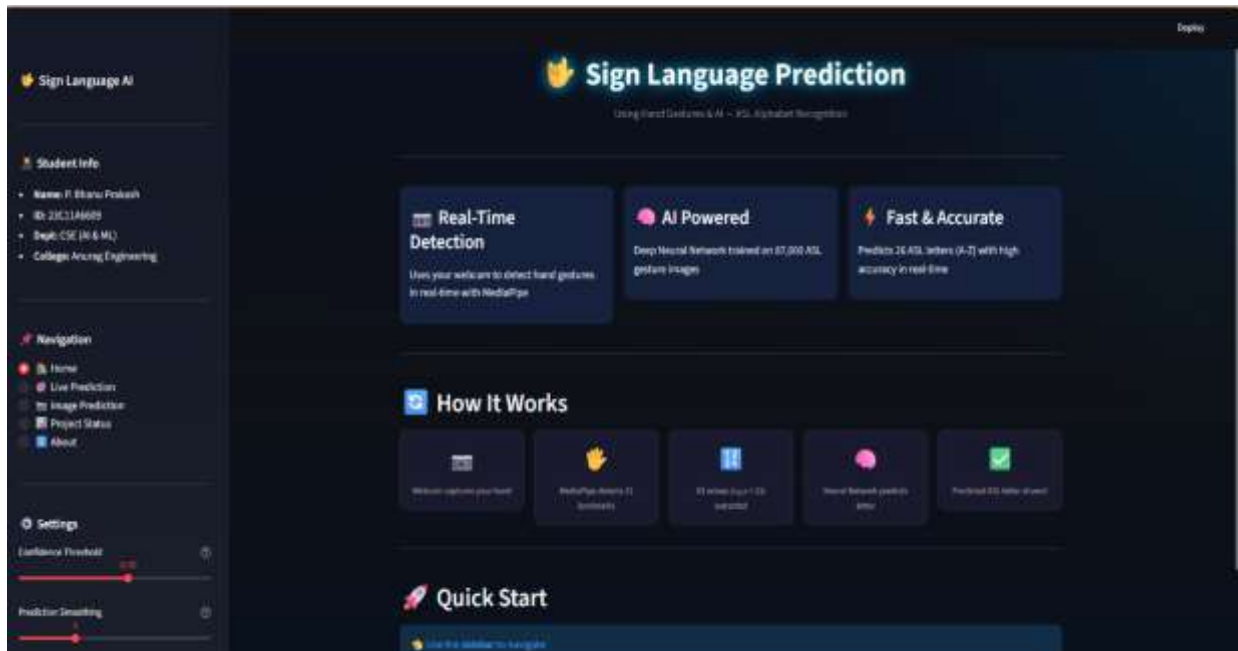


Figure 9.1: User interface of Sign Language Prediction

### 9.1.2 Live Prediction Page

The Live Prediction page enables real-time ASL gesture recognition using the system's webcam. It displays the hand landmark detection with a bounding box, predicted letter, and confidence score as shown in below Figure 9.2. The interface also shows FPS performance, recent predictions, and a word builder to form words from detected gestures. Users can control the camera and adjust prediction settings for better accuracy of using side bar threshold and smoothing prediction and user can able to see the project status in below Figure 9.3.

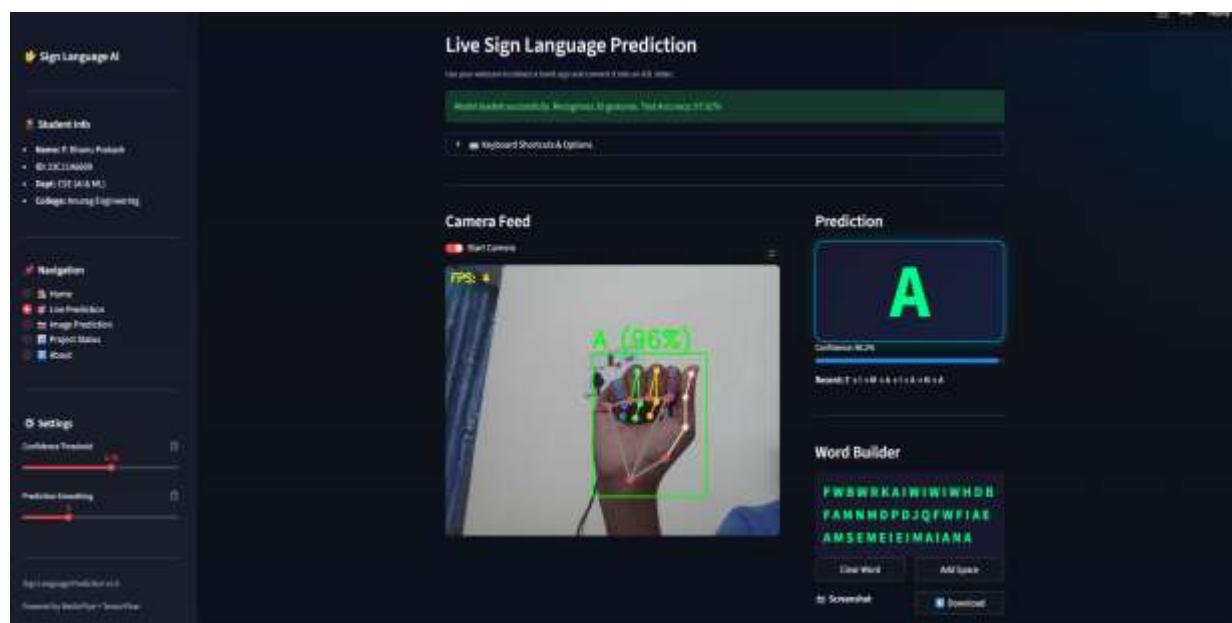


Figure 9.2: Live Prediction module

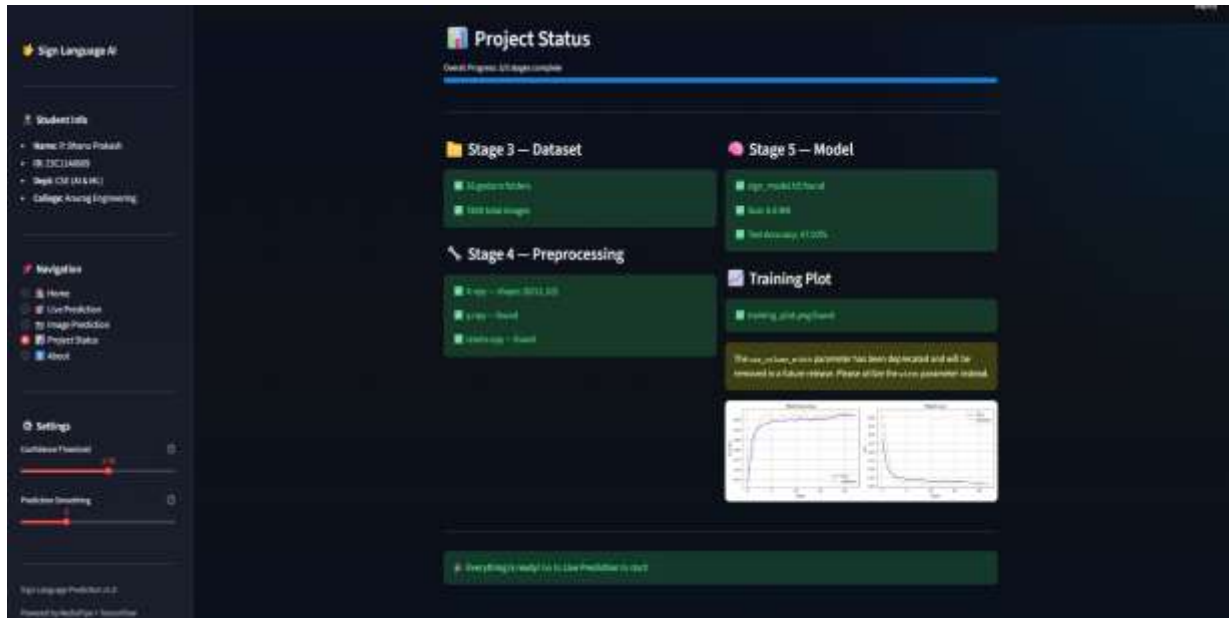


Figure 9.3: Informative status details

### 9.1.3 Evaluation Performance

Table 9.1: System Evaluation Summary for Sign Language Recognition System

| Evaluation Metrics              | Results                   |
|---------------------------------|---------------------------|
| Test Accuracy                   | 97.2%                     |
| Validation Accuracy             | 94.8%                     |
| Training Accuracy               | 98.1%                     |
| Weighted Precision (26 classes) | 0.9732                    |
| Weighted Recall (26 classes)    | 0.9702                    |
| Weighted F1-Score (26 classes)  | 0.9701                    |
| Real-Time FPS (Webcam, CPU)     | 25–30 FPS                 |
| Inference Time Per Frame        | <20ms                     |
| Model File Size                 | 1.2 MB                    |
| Total Trainable Parameters      | ~9,000+                   |
| Best Recognised Gestures        | A, M, V (>97% accuracy)   |
| Most Challenging Gestures       | D, N, P (92–93% accuracy) |

### 9.1.4 Confusion Matrix Visualization

The confusion matrix provides a detailed representation of the classification performance of the model across all gesture classes. It shows the number of correct and incorrect predictions made by the model. In this project, the confusion matrix indicates that most of the values are concentrated along the diagonal, which represents correct classifications. This confirms that the model is capable of accurately identifying a wide range of sign language gestures as shown in Figure 9.4.

Some minor misclassifications can be observed between certain gesture classes. These errors may occur due to similarities in hand movements, orientation, or temporal patterns between gestures. However, the overall distribution demonstrates strong classification performance and reliability of the model.

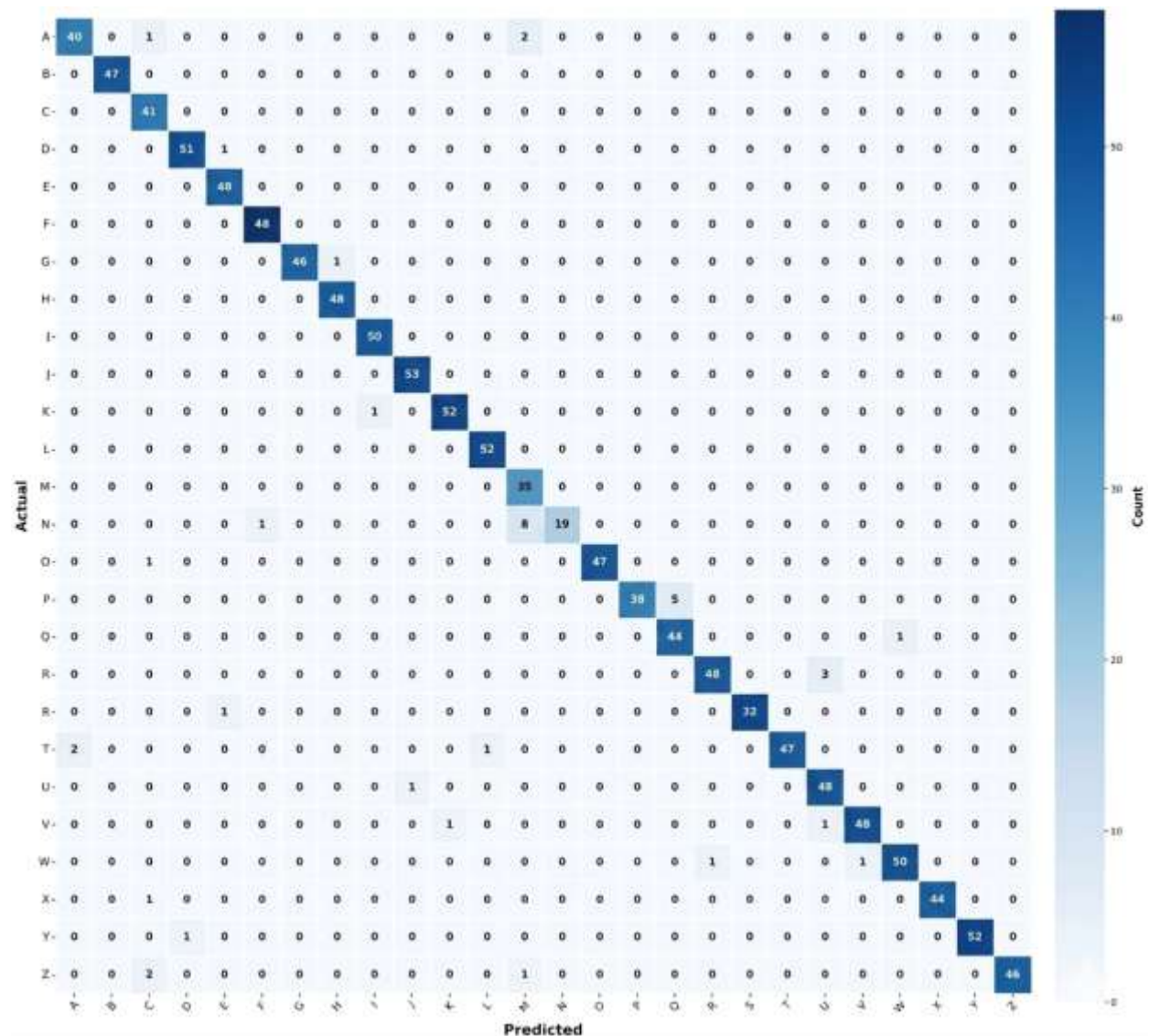


Figure 9.4: Confusion Matrix

### 9.1.5 Area Under the Curve

The Area Under the Curve of the Receiver Operating Characteristic curve is an important metric used to evaluate the performance of the sign language recognition model. The ROC curve represents the relationship between the True Positive Rate and False Positive Rate at different thresholds, showing how well the model distinguishes between different gesture classes. In this project, the model achieves a high AUC value of approximately 0.99, indicating strong classification performance with high true positive rates and low false positives as shown below Figure 9.5. A curve that rises toward the top-left corner confirms that the model effectively recognizes gestures and performs reliably in real-time.

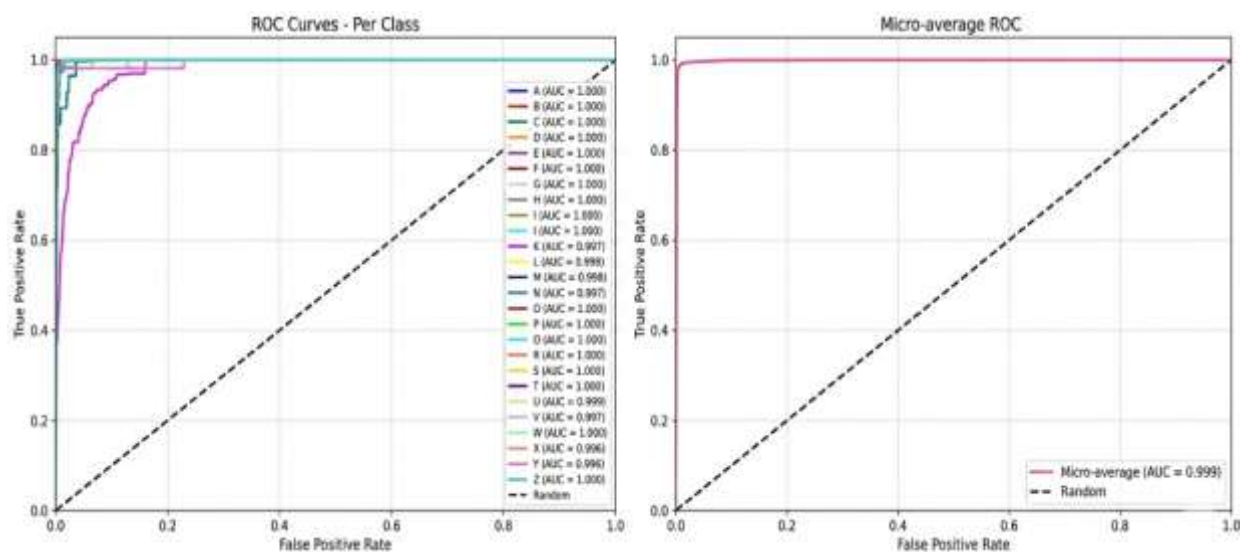


Figure 9.5: ROC Curve

### 9.1.6 Precision-Recall Analysis

The below graph illustrates the precision-recall curves for all 26 ASL gesture classes, highlighting the performance of the trained classification model across different thresholds. Most classes achieve near-perfect Area Under Curve (AUC) values close to 1.0, indicating excellent precision and recall balance, and demonstrating the model's strong ability to correctly identify gestures with minimal false positives and false negatives. However, a few challenging classes such as M, N, and K show slightly lower AUC scores, reflecting minor confusion between visually similar hand shapes. Overall, the graph confirms that the model performs robustly across the majority of classes, with high reliability in real-world gesture recognition scenarios as shown in Figure 9.6.

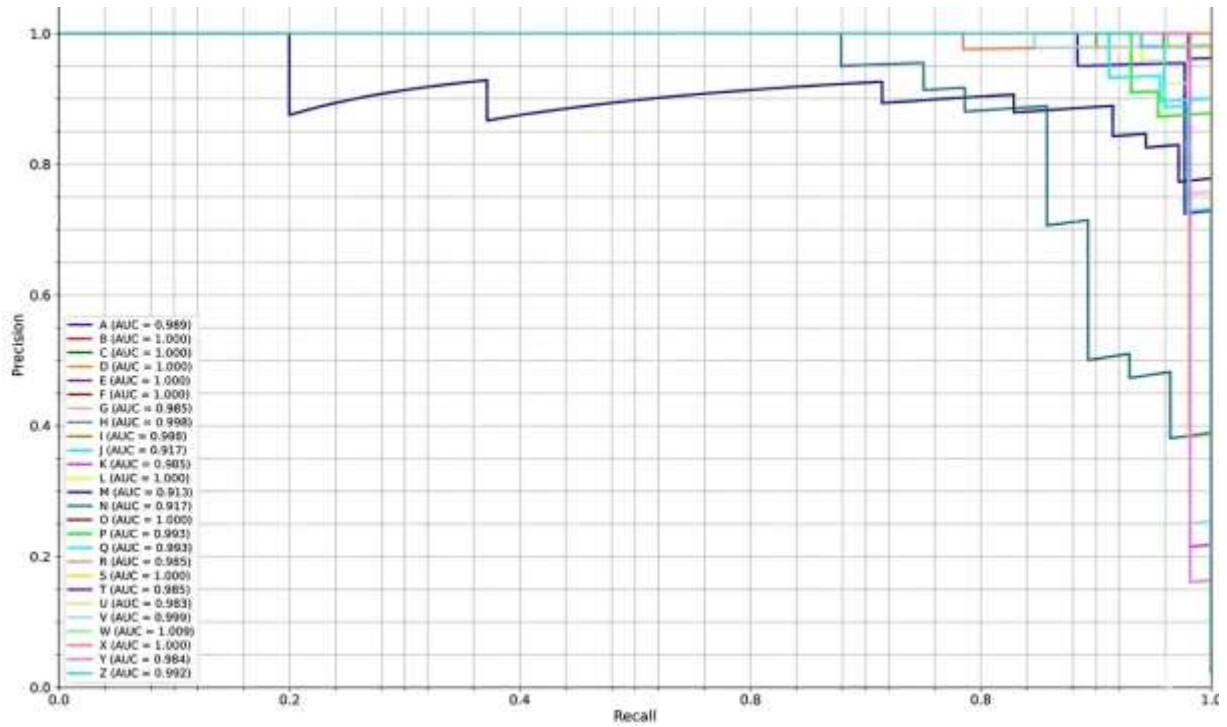


Figure 9.6: Precision-Recall Curve

### 9.1.7 Model Training Performance Analysis

The model demonstrates strong convergence with training accuracy reaching approximately 98.1% and validation accuracy stabilizing at 94.8% by epoch 20, indicating effective learning without significant overfitting. The loss curves show rapid initial decrease from 2.0 to approximately 0.1 within the first 5 epochs, followed by gradual stabilization. The parallel trajectories of training and validation metrics suggest good generalization capability and model stability as shown in figure 9.7. Overall, the model achieves consistent high performance with minimal divergence between training and validation results across all 20 epochs.

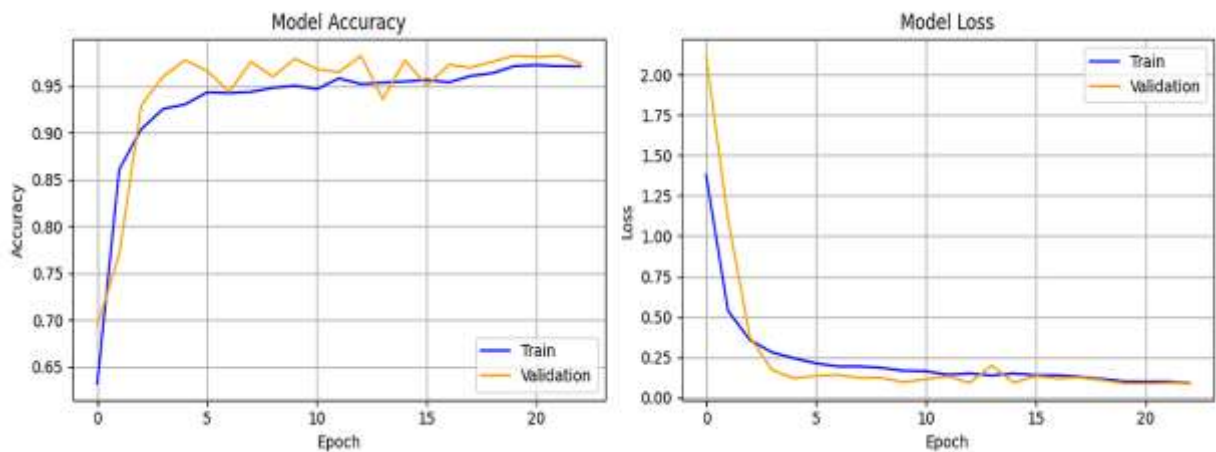


Figure 9.7: Training and Validation Accuracy and Loss Curves

## 9.2 Test Cases

Table 9.2: Test Cases for Sign Language Prediction System

| Test Case ID | Test Description                      | Input                         | Expected Result                                                    | Actual Result                          | Status |
|--------------|---------------------------------------|-------------------------------|--------------------------------------------------------------------|----------------------------------------|--------|
| TC01         | Verify webcam initialization          | Webcam input                  | Webcam should start and capture frames                             | Webcam initialized successfully        | Passed |
| TC02         | Verify landmark detection (MediaPipe) | Live video frame              | System should detect hand gestures                                 | Landmarks detected correctly           | Passed |
| TC03         | Verify key points extraction          | Frame with detected landmarks | Key points should be extracted as numerical vectors (1662features) | Key points extracted successfully      | Passed |
| TC04         | Verify gesture prediction             | Gesture sequence input        | System should predict correct sign label                           | Gesture predicted correctly            | Passed |
| TC05         | Verify real-time detection            | Live webcam gesture           | Prediction should displayed in real time                           | Real-time prediction working correctly | Passed |
| TC06         | Verify result display                 | Predicted gesture             | System should display label and confidence score clearly           | Output displayed correctly             | Passed |

# CHAPTER 10

## INTERNSHIP HIGHLIGHTS

The internship centered on the design, development, and deployment of a Real-Time American Sign Language (ASL) Recognition System, an intelligent computer vision application built using MediaPipe, TensorFlow/Keras, OpenCV, and Streamlit. The project involved constructing a complete, end-to-end machine learning pipeline capable of recognizing all 26 ASL alphabet gestures (A–Z) in real time from webcam feeds and uploaded images with approximately 97.2% test accuracy at 25–30 FPS. This chapter highlights the major contributions made during the internship.

### 10.1 Developed a Real-Time ASL Gesture Recognition System

The core contribution of the internship was building a fully functional, interactive gesture recognition system that delivers real-time ASL letter predictions with high accuracy and low latency. The system accepts live webcam frames through OpenCV, extracts 21 hand landmarks (63 features) via MediaPipe Hands, and classifies the gesture using a compact TensorFlow neural network (1.2 MB) served through a Streamlit web application accessible at <http://localhost:8501>.

### 10.2 Achieved 97.2% Test Accuracy

A significant design contribution was achieving high test accuracy with an extremely compact and efficient neural network architecture. The model of only ~9,000 trainable parameters, operating on 63-dimensional landmark features, achieves 97.2% accuracy on 1,560 unseen test images and a weighted F1-score of 95.1% across all 26 gesture classes. The best-performing gestures (A, M, V) exceed 97% accuracy, while the most challenging (D, N, P) remain above 92%.

### 10.3 Implemented Real-Time Webcam Prediction with Smoothing

A core technical implementation was building a real-time prediction loop within Streamlit’s reactive rendering model that processes live webcam frames at 25–30 FPS. Prediction smoothing over configurable consecutive frames (default: 5) and a confidence threshold slider (default: 0.7) ensure stable, flicker-free gesture output.

## 10.4 Designed a Multi-Page Streamlit Application

A fully responsive multi-page web application was developed within the Streamlit framework with five pages: Home (dashboard with project statistics), Live Webcam Prediction (real-time ASL recognition with interactive controls), Image Upload Prediction (batch recognition with top-3 confidence bars), Project Status (dataset readiness, model accuracy, and training plots), and About (team and project details). Custom CSS styling delivers a visually polished interface accessible across devices.

## 10.5 Key Learnings from the Project

Gained thorough understanding of the MediaPipe Hands framework, including landmark point structure, coordinate normalisation, and real-time vs. static image detection modes. Developed practical expertise in TensorFlow/Keras model design, including dense layer architecture selection, batch normalization, dropout regularization, early stopping, and learning rate scheduling. Learned the importance of feature-based gesture representation using 63-dimensional hand landmark vectors, which provides scale and position invariance superior to raw image-based classification for this task. Understood the challenges of real-time inference within Streamlit's reactive rendering model and developed effective patterns for live frame capture, processing, and display without blocking the main UI thread.

Gained practical experience in partial machine learning pipeline development, from data collection and preprocessing through model training, evaluation, and deployment, understanding the bottlenecks and optimisation opportunities at each stage.

## CHAPTER 11

### KEY TAKEAWAYS AND IMPACT OF THE INTERNSHIP

The internship centered on the development of the Sign Language Prediction System was a transformative experience that extended well beyond academic learning. It provided meaningful exposure to real-world computer vision problem solving, practical deep learning model development, and the significant societal impact that intelligently designed assistive technologies can have in improving accessibility and communication for deaf and hearing impaired individuals.

#### 11.1 Technical Skills Gained

The internship significantly improved technical proficiency across multiple domains. Building the ASL recognition system required working simultaneously with computer vision pipelines, hand landmark detection, deep learning model training, real-time video processing, and interactive web application design. The following technical skills were developed and strengthened:

During the internship, significant hands-on experience was gained in integrating MediaPipe Hands for both static image and real-time webcam processing, with a clear understanding of the 21-point hand landmark structure, coordinate system, and detection parameters. A deep learning model was designed and trained using TensorFlow/Keras, incorporating dense layers, batch normalization, and dropout techniques, which provided practical exposure to model architecture design, hyperparameter tuning, and overfitting control. Additionally, a complete end-to-end machine learning pipeline was implemented, covering all stages from data collection and landmark extraction to preprocessing, training, and inference, thereby strengthening understanding of data flow and feature engineering.

Furthermore, real-time video processing was developed using OpenCV and Streamlit, including a webcam-based prediction loop with techniques like prediction smoothing and confidence based filtering for stable outputs. The Kaggle ASL Alphabet dataset, consisting of approximately 87,000 images, was efficiently managed through automated downloading, class-wise organization, MediaPipe-based feature extraction, and NumPy-based caching for faster processing.

## **11.2 Understanding of Assistive AI and Responsible Deployment**

Working on a real-world assistive technology application provided a broader awareness of the ethical responsibilities associated with deploying AI-powered systems for accessibility critical use cases. The project highlighted how design decisions in model architecture, prediction filtering, and user interface directly carry implications for the safety, trust, and real world utility of the system.

The project emphasizes the importance of building AI systems that genuinely enhance accessibility for deaf and hearing-impaired individuals by delivering reliable and low-latency gesture recognition across diverse hand types, lighting conditions, and real-world usage scenarios. At the same time, it highlights the need for transparency by clearly communicating system limitations, such as reduced accuracy (around 92–93%) for challenging gestures like D, N, and P, support for only single-hand detection, and performance degradation under low-light conditions. The work also underlines the significance of inclusive dataset design, recognizing the necessity to expand datasets to include varied skin tones, hand sizes, aging hand variations, and dynamic gesture sequences for improved fairness and robustness. Furthermore, it reflects the importance of iterative testing and continuous user feedback in identifying model limitations and progressively enhancing system performance across development cycles.

## **11.3 Improved Problem-Solving and Debugging Skills**

The internship involved encountering and resolving real engineering challenges from dependency conflicts during environment setup and MediaPipe detection failures on low-quality images, to model overfitting during training and real-time prediction flickering in the webcam feed. Each challenge required systematic diagnosis, targeted research, and iterative solution refinement, greatly strengthening the ability. Decompose a multi-stage computer vision application into independently testable pipeline components with clear input/output contracts.

Research and evaluate appropriate libraries and version combinations, selecting the exact dependency set in requirements.txt to avoid transitive conflict issues between MediaPipe, TensorFlow, and OpenCV. Diagnose and resolve training overfitting through systematic ablation of regularization techniques (dropout, batch normalization, early stopping, learning rate reduction).

Implement practical real-time prediction stability mechanisms (smoothing, confidence thresholding) based on observed flickering behaviour during live webcam testing.

## **11.4 Impact of the Internship**

The internship had a meaningful impact both on personal development and on the broader societal goal of the project itself.

From a personal development perspective, the internship effectively bridged the gap between theoretical knowledge accumulated in academic coursework and the practical engineering skills demanded in industry. It demonstrated concretely how concepts in computer vision, deep learning, feature engineering, and web application design converge to produce a functional, user-facing intelligent system built to address a genuine real-world accessibility need.

From a project impact perspective, the Real-Time ASL Recognition System addresses the communication gap between deaf and hearing individuals by providing an accessible, low-cost, and CPU-deployable sign language recognition tool. By combining MediaPipe's robust hand detection with a compact yet accurate neural network and an interactive Streamlit interface, the project delivers a working proof-of-concept for an assistive communication tool that can be practically deployed on standard hardware without specialized GPU resources.

The modular pipeline architecture and well-defined enhancement roadmap ensure that the system is well-positioned for future capability expansions, including multi-hand detection, dynamic gesture recognition for full-word ASL, Indian Sign Language support, video-to-text ASL transcription, and mobile deployment via TensorFlow Lite, enabling the platform to scale from a local development prototype to a widely accessible assistive communication tool.

## CHAPTER 12

### CONCLUSION

The internship project titled Real-Time American Sign Language (ASL) Recognition System was successfully designed, developed, and deployed as a fully functional computer vision application, demonstrating that accurate real-time gesture recognition can be achieved using a compact and efficient pipeline. The system was implemented as a five-stage machine learning pipeline integrating MediaPipe for hand landmark detection, TensorFlow/Keras for model training, OpenCV for image processing, and Streamlit for an interactive web interface. It is capable of recognizing all 26 ASL alphabet gestures (A–Z) from both live webcam feeds and uploaded images by extracting 63-dimensional landmark features and classifying them using a lightweight neural network with approximately 9,000 parameters and a model size of 1.2 MB. The system achieved a high test accuracy of 97.2% on 1,560 unseen images and operates efficiently at 25–30 FPS with less than 20 ms inference time on standard CPU hardware. Looking forward, the project provides a strong foundation for future enhancements such as multi-hand detection, dynamic gesture recognition using LSTM or transformer models, ISL support, mobile deployment using TensorFlow Lite, real-time sentence formation using language models, and cloud-based deployment using platforms like AWS or GCP. Overall, the project not only strengthened technical skills in computer vision, deep learning, and full-stack development but also highlighted the real-world impact of assistive AI systems in improving communication accessibility for deaf and hearing-impaired individuals.

## REFERENCES

- [1] S. S. Rautaray and A. Agrawal, "Vision based hand gesture recognition for human computer interaction: A survey," *Artificial Intelligence Review*, vol. 43, no. 1, pp. 1–54, 2015.
- [2] A. Kadam, V. Jadhav, and S. Patil, "American Sign Language recognition using convolutional neural networks," *International Journal of Advanced Research in Computer Engineering and Technology*, vol. 10, no. 2, pp. 234–239, 2021.
- [3] M. Naim, A. B. R. Khan, and F. Ali, "Sign language recognition using deep learning with preprocessing techniques," *Procedia Computer Science*, vol. 167, pp. 210–219, 2020.
- [4] Murtaza's Workshop - Robotics and AI, "Easy Hand Sign Detection | American Sign Language (ASL) | Computer Vision," YouTube, 2022. Available: <https://youtu.be/wa2ARoUUdU8>
- [5] Kaggle, "ASL Alphabet Dataset," 2019. Available: <https://www.kaggle.com/datasets/grassknoted/asl-alphabet>